

06026240 การพัฒนาระบบอัจฉริยะ

LAB 7B

สัปดาห์การเรียนรู้จากเกมหลักสูตร

ด้วย LLM ที่รันบนเครื่องตัวเอง — ออฟไลน์ ฟรี ทำซ้ำได้

| หัวข้อ        | รายละเอียด                                             |
|---------------|--------------------------------------------------------|
| กลุ่มเป้าหมาย | นักศึกษาโครงการ "ระบบถาม-ตอบหลักสูตร (Curriculum Q&A)" |

1. วัตถุประสงค์การเรียนรู้

เมื่อทำแล็บนี้เสร็จ นักศึกษาจะสามารถ:

- ติดตั้งและเรียกใช้ Vision-Language Model บนเครื่องของตนเองได้ โดยไม่มีค่าใช้จ่าย
- ตัดสินใจได้ว่าเอกสารใดควรใช้ OCR และเอกสารใดควรดึงข้อความจาก PDF โดยตรง
- จัดการเอกสารที่ยาวเกิน context window ด้วยการแบ่งก้อน (chunking) และรวมผลอย่างถูกต้อง
- ออกแบบ JSON Schema และใช้ structured output บังคับรูปแบบคำตอบ
- ออกแบบกฎตรวจสอบความถูกต้องเชิงโครงสร้าง เช่น referential integrity ของวิชาบังคับก่อน
- คำนวณและตีความ CER / WER แยกราย attribute และอธิบายบทบาทของ normalization
- ตัดคำภาษาไทยด้วย PyThaiNLP
- ตรวจสอบคุณภาพของ ground truth ก่อนนำมาใช้วัดผล



ทำไมแล็บนี้จึงสำคัญต่อโครงการของคุณ

ระบบ RAG ที่จะสร้างใน Lab ต่อไป ตอบคำถามได้ดีเท่าที่ข้อมูลตั้งต้นถูกต้องเท่านั้น ถ้าหน่วยกิตในฐานข้อมูลผิด ระบบจะตอบผิดอย่างมั่นใจ ซึ่งอันตรายกว่าการตอบว่าไม่รู้ หลักการนี้เรียกว่า garbage in, garbage out และไม่มีเทคนิค RAG ไດแก้ได้

2. ส่วนที่ 1 — ติดตั้งโมเดลบนเครื่องตัวเอง (15 นาที)



ของที่มีอยู่แล้วจาก Lab ก่อนหน้า ไม่ต้องติดตั้งใหม่

Tesseract พร้อมภาษาไทย · OpenCV · NumPy — ติดตั้งไปแล้วในแล็บก่อนหน้า  
virtual environment เดิม — ใช้ตัวเดิมได้เลย ไม่ต้องสร้างใหม่ (เหตุผลอยู่ในหัวข้อ 2.4)  
ตรวจว่า Tesseract ยังใช้ได้ (ถ้าอยาก): `tesseract --list-langs` ต้องเห็นคำว่า tha ในรายการ

2.1 ทำไมต้องรันบนเครื่องตัวเอง

กลุ่มของคุณทำงานกับเอกสารสาธารณะ จึงไม่มีข้อจำกัดด้าน PDPA แต่ยังมีเหตุผลอีกสองข้อที่สำคัญไม่แพ้กัน

- ค่าใช้จ่าย — เกมหลักสูตรมีขนาดใหญ่มาก การส่งทั้งเล่มเข้า API เชิงพาณิชย์ฯ ะหว่างการทดลองปรับพารามิเตอร์ จะมียกจ่ายสูงอย่างรวดเร็ว และแล็บนี้กำหนดว่านักศึกษาต้องไม่มีค่าใช้จ่าย

2. ความสามารถทำซ้ำได้ — ผู้ให้บริการ API เปลี่ยนรุ่นโมเดลเบื้องหลังได้ตลอดเวลาโดยไม่แจ้ง ทำให้ผลการทดลองเมื่อสามเดือนก่อนทำซ้ำไม่ได้ ซึ่งเป็นปัญหาร้ายแรงเมื่อนำไปเขียนรายงานหรืองานวิจัย ส่วนโมเดลที่เก็บไว้ในเครื่องจะไม่เปลี่ยนไปเอง

2.2 ติดตั้ง Ollama

Ollama คือโปรแกรมที่ทำให้การรันโมเดลภาษาบนเครื่องง่ายพอ ๆ กับติดตั้งแอปทั่วไป เมื่อติดตั้งแล้วจะเปิดบริการรออยู่ที่ `http://127.0.0.1:11434` โดยเลข 127.0.0.1 คือ loopback address หมายถึง "ตัวเครื่องเอง" ข้อมูลที่ส่งไปที่นี้จะไม่ผ่านการคัดกรองใดๆเลย

| ระบบปฏิบัติการ  | ขั้นตอน                                                                                                                                                                                                                                            |
|-----------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Windows 10 / 11 | เข้า <a href="https://ollama.com/download">https://ollama.com/download</a> เลือก Windows → ดาวน์โหลด<br>OllamaSetup.exe → ติดตั้ง (ไม่ต้องใช้สิทธิ์ผู้ดูแลระบบ)<br>เมื่อเสร็จจะมีไอคอนที่ system tray มุมขวาล่าง และ service จะเริ่มทำงานอัตโนมัติ |
| macOS           | ดาวน์โหลด Ollama.dmg → ลาก Ollama.app ไป Applications → เปิดครั้งแรก<br>กดยืนยันติดตั้งคำสั่ง command line<br>หรือถ้ามี Homebrew: <code>brew install ollama</code> แล้ว <code>brew services start ollama</code>                                    |
| Linux / WSL2    | <code>curl -fsSL https://ollama.com/install.sh   sh</code><br>บน WSL2 ต้องเปิด service เองในอีก terminal: <code>ollama serve</code><br>(อย่าปิดหน้าต่างนั้นระหว่างทำแล็บ)                                                                          |

 **ตรวจว่า service ทำงานอยู่จริง**

เปิดเบราว์เซอร์ไปที่ `http://127.0.0.1:11434` ถ้าเห็นข้อความ "Ollama is running" แสดงว่าพร้อมแล้ว

ถ้าไม่ขึ้น ให้เปิด terminal แล้วสั่ง `ollama serve` ทิ้งไว้

2.3 ดาวน์โหลดโมเดล 2 ตัว

รวมประมาณ 5.5 GB ควรทำล่วงหน้าเมื่ออยู่บนเครือข่ายที่เร็ว ทั้งสองตัวทำงานคนละหน้าที่และขาดตัวใดตัวหนึ่งไม่ได้

```
# — 1. อ่านภาพเอกสารภาษาไทย → Markdown (~3 GB) —————
# Typhoon-OCR จาก SCB 10X fine-tune จาก Qwen-VL มาสำหรับเอกสารไทย
# ⚠ เป็น task-specific model: รับผิดชอบเฉพาะ prompt ของตัวเอง
# สั่งให้พ่น JSON ตาม schema ของเราไม่ได้ จึงต้องมีตัวที่ 2
ollama pull scb10x/typhoon-ocr1.5-3b
# — 2. Markdown → JSON ตาม schema (~2.5 GB) —————

# เครื่องแรม 8 GB ให้เปลี่ยนเป็น qwen3:1.7b
# เครื่องแรม 32 GB ขึ้นไป ใช้ qwen3:8b จะได้ผลดีกว่ามากสำหรับตารางยาว
ollama pull qwen3:4b

ollama list      # ตรวจว่าครบทั้งสองตัว
```

**i** ถ้าชื่อโมเดลเปลี่ยนไป

หาก ชื่อ tag เปลี่ยน หรือ อาจต่างจากที่เขียนไว้ ให้ค้นชื่อล่าสุดที่ <https://ollama.com/search?c=vision> แล้วตั้งค่าผ่าน environment variable โดยไม่ต้องแก้โค้ด:

```
export LAB7_MODEL_OCR="scb10x/typhoon-ocr-3b"
```

## 2.4 โลบารรี Python — ใช้ environment เดิมของ Lab ก่อนหน้า

**✓** ไม่ต้องสร้าง virtual environment ใหม่

Ollama ไม่ใช่แพ็คเกจ Python แต่เป็นโปรแกรมระดับระบบปฏิบัติการที่รันเป็น service แยกต่างหาก โค้ด Python ของเราเพียงส่งคำขอ HTTP ไปหาที่ 127.0.0.1:11434 เท่านั้น

โมเดล LLM ทั้งหมดจึงอยู่ "นอก" virtual environment โดยสิ้นเชิง ไม่มีทางชนกับแพ็คเกจใด ๆ ได้

โลบารรีที่ต้องเพิ่มมีเพียง 5 ตัว และไม่มีตัวใดขัดแย้งกับ OpenCV หรือ Tesseract ที่ติดตั้งไปแล้ว

```
# เข้า environment เดิมที่ใช้ใน Lab ก่อนหน้า
source .venv/bin/activate      # Windows: .venv\Scripts\activate

pip install requests pymupdf pillow pdfplumber pythainlp

# requests → เรียก Ollama ผ่าน HTTP
# pymupdf → แปลง PDF เป็นภาพ (import ด้วยชื่อ fitz)
# pillow → จัดการภาพ
# pdfplumber → ดึงข้อความจาก PDF โดยตรง (pipeline "text")
# pythainlp → ★ ตัดคำไทย จำเป็นมากสำหรับกลุ่มนี้ ใช้คำนวณ WER
```

## 2.5 ตรวจสอบความพร้อมด้วยสคริปต์

```
python3 lab7b_curriculum.py --check
```

ผลลัพธ์ที่ควรได้:

- ✓ พบ Ollama: ollama version 0.x.x
  - ✓ Ollama service ทำงานที่ http://127.0.0.1:11434
  - ✓ [อ่านภาพ] scb10x/typhoon-ocr1.5-3b
  - ✓ [จัด JSON] qwen3:4b
  - ✓ python: fitz / PIL / requests / pdfplumber
  - ✓ tesseract (จาก Lab ก่อนหน้า)
  - ✓ pythainlp (ตัดคำไทยสำหรับ WER ★)
- พร้อมใช้งาน ✓

## 2.6 ทางเลือก: รันบน Google Colab

กลุ่มของคุณทำงานกับเอกสารสาธารณะ จึงใช้ Colab ได้ ซึ่งช่วยได้มากถ้าเครื่องส่วนตัวไม่แรง ใส่โค้ดต่อไปนี้ในเซลล์แรกของ notebook

```
!curl -fsSL https://ollama.com/install.sh | sh

import subprocess, time
subprocess.Popen(["ollama", "serve"],
                 stdout=subprocess.DEVNULL, stderr=subprocess.DEVNULL)
time.sleep(8)                # รอให้ service พร้อม

!ollama pull scb10x/typhoon-ocr1.5-3b
!ollama pull qwen3:4b
!pip install -q requests pymupdf pillow pdfplumber pythainlp
!python3 lab7b_curriculum.py --check
```

### ⚠️ ข้อจำกัดของ Colab ที่ต้องรู้

Colab ตัดการเชื่อมต่อเมื่อไม่มีการใช้งาน ต้องดาวน์โหลดโมเดลใหม่ทุกครั้งที่เราเริ่ม session ให้เลือก Runtime → Change runtime type → T4 GPU ก่อนเสมอ ไม่งั้นจะช้ามาก

บันทึกไฟล์ output ลง Google Drive ทันทีที่ได้ผล ไม่งั้นหายเมื่อ session จบ

ถ้าโครงการของคุณมีเอกสารภายในที่ยังไม่เผยแพร่ ห้ามอัปโหลดขึ้น Colab

## 2.7 ความต้องการของเครื่อง

| สเปกเครื่อง               | ชุดโมเดลที่แนะนำ               | ความเร็วโดยประมาณ |
|---------------------------|--------------------------------|-------------------|
| RAM 8 GB, ไม่มี GPU       | typhoon-ocr1.5-2b + qwen3:1.7b | 5-10 นาที/หน้า    |
| RAM 16 GB, ไม่มี GPU      | ชุดมาตรฐานของแล็บ              | 1-3 นาที/หน้า     |
| GPU 8 GB VRAM ขึ้นไป      | ชุดมาตรฐานของแล็บ              | 10-30 วินาที/หน้า |
| Mac (Apple Silicon) 16 GB | ชุดมาตรฐานของแล็บ              | 20-60 วินาที/หน้า |
| Google Colab (T4 ฟรี)     | ชุดมาตรฐานของแล็บ              | 15-40 วินาที/หน้า |

### ⚠️ คำนวณเวลาก่อนลงมือ

ถ้าเล่มหลักสูตรมี 120 หน้า และเครื่องใช้เวลา 2 นาทีต่อหน้า จะใช้เวลา 4 ชั่วโมง

นี่คือเหตุผลที่แล็บนี้บังคับให้ใช้ตัวเลือก --pages เลือกเฉพาะหน้าที่เป็นตารางแผนการศึกษา

โดยทั่วไปตารางแผนการศึกษายู่ประมาณ 10-20 หน้าเท่านั้น

### 3. ทำความเข้าใจเส้นทางการที่จะเปรียบเทียบ

#### 3.1 ตรวจสอบว่า PDF ของคุณเป็นชนิดไหน

##### ✓ บทเรียนที่สำคัญที่สุดของส่วนนี้

เครื่องมือที่ทันสมัยที่สุด ไม่ใช่เครื่องมือที่ดีที่สุดเสมอไป

ถ้า PDF มีข้อความฝังอยู่แล้ว การเอาไปทำ OCR คือการทำงานซ้ำซ้อน และยังทำให้ผลแย่ลง

เพราะ OCR มีโอกาสอ่านตัวอักษรผิด แต่ข้อความที่ฝังมากับไฟล์ไม่มีทางผิด

ก่อนเลือกเครื่องมือ ต้องเข้าใจปัญหาที่แท้จริงก่อนเสมอ

ทดสอบด้วยโค้ดสั้น ๆ นี้

```
import pdfplumber

with pdfplumber.open("data/DSBA.pdf") as pdf:
    for i in range(min(5, len(pdf.pages))):
        txt = pdf.pages[i].extract_text() or ""
        print(f"หน้า {i+1}: {len(txt):,} ตัวอักษร")

# ตีความผล:
# > 500 ตัวอักษรต่อหน้า → digital PDF ใช้ pipeline "text" ได้เลย
# < 100 ตัวอักษรต่อหน้า → scanned PDF ต้องใช้ OCR
# ถ้ากึ่ง → ไฟล์ผสม บางหน้าสแกน บางหน้าไม่ ต้องแยกจัดการ
```

##### ⚠ กับดักที่พบบ่อยมากในเล่มหลักสูตร

extract\_text() แบบธรรมดา "ทำตารางพัง" เพราะมันเรียงข้อความตามลำดับในไฟล์ ไม่ใช่ตามตำแหน่งบนหน้า

ผลคือคอลัมน์ปนกันมั่ว ทำให้ปี/ภาคของแต่ละวิชาผิดหมด

ต้องใช้ extract\_text(layout=True) ซึ่งรักษาระยะห่างแนวนอนไว้

สคริปต์ของเล่มนี้ใช้ layout=True ให้แล้ว — ลองเปลี่ยนเป็นแบบธรรมดาเพื่อดูเห็นความต่าง

#### 3.2 ปัญหาใหญ่ของกลุ่มนี้: เอกสารยาวเกิน context window

เล่มหลักสูตร 150 หน้า เมื่อแปลงเป็น token จะประมาณ 200,000 token

ซึ่งเกินความสามารถของโมเดลขนาดเล็กที่รันบนเครื่องได้ทั่วไป (มัก 32,000–128,000 token) และแม้ context จะพอ

ก็ยังเจอปัญหาอีกสามอย่าง

| ปัญหา                         | อาการที่สังเกตได้                                                         |
|-------------------------------|---------------------------------------------------------------------------|
| เกิน context window           | โมเดล "ตัดท้ายทิ้งเจิบ ๆ" ไม่มีข้อความเตือน<br>สังเกตจากวิชาปี 4 หายไปหมด |
| output ยาวเกิน<br>num_predict | JSON ขาดกลางคัน parse ไม่ได้<br>สคริปต์จะเตือนว่า "ผลลัพธ์อาจถูกตัด"      |
| Lost in the middle            | โมเดลจำข้อมูลตรงต้นและท้ายได้ดี แต่ลืมตรงกลาง                             |

| ปัญหา        | อาการที่สังเกตได้                                                             |
|--------------|-------------------------------------------------------------------------------|
|              | เป็นปรากฏการณ์ที่พบในงานวิจัยหลายชิ้น                                         |
| ใช้แรมมหาศาล | num_ctx 32768 กินแรมเพิ่ม 2–3 GB<br>เครื่องแรมน้อยจะสลับไปใช้ swap แล้วช้ามาก |

#### ทางแก้: แบ่งก้อน (chunking)

```
# ค่าเริ่มต้นคือ 3 หน้าต่อก่อน ปรับได้ผ่าน environment variable
LAB7_CHUNK=2 python3 lab7b_curriculum.py -i data/x.pdf -p vlm
```

การแบ่งก้อนแลกมาด้วยราคาที่ต้องจ่าย: โมเดลไม่เห็นภาพรวมทั้งเล่ม เช่น อาจไม่รู้่วิชานี้อยู่หมวดใด

ถ้าหัวข้อหมวดอยู่คนละก้อน วิธีแก้ในระบบจริงคือ

- ทำให้ก้อนซ้อนทับกัน (overlap) หนึ่งหน้า เพื่อไม่ให้ข้อมูลที่คร่อมรอยต่อหายไป
- ส่งข้อมูลบริบทที่จำเป็น เช่น ชื่อหมวดวิชาที่เจอล่าสุด ไปพร้อมกับก้อนถัดไป
- ทำสองรอบ รอบแรกอ่านหัวข้อทั้งหมด รอบที่สองอ่านรายวิชาโดยมีหัวข้อเป็นบริบท

การนำสองข้อแรกไปทำจริงคือโจทย์ท้าทายข้อ 1

### 3.3 Structured Output — บังคับรูปร่างของคำตอบ

การเขียนในพรมอบว่า "ตอบเป็น JSON เท่านั้น" เป็นเพียง "คำขอ" ไม่ใช่ "การบังคับ" โมเดลอาจใส่คำนำหน้า ใส่ markdown fence หรือถูกตัดกลางคัน

Ollama รองรับการส่ง JSON Schema ผ่าน field ชื่อ format ซึ่งจะควบคุมขั้นตอนถอดรหัส (constrained decoding)

ให้เลือกได้เฉพาะ token ที่ยังทำให้ JSON ถูกต้องตาม schema

```
payload = {
  "model": "qwen3:4b",
  "messages": [...],
  "stream": False,
  "format": COURSE_SCHEMA,      # <-- ตรงนี้คือหัวใจ
  "options": {"temperature": 0, "num_ctx": 32768},
}
```

### 3.4 การออกแบบ Prompt

พรมบ์ในสคริปต์มี 9 กติกา แต่ละข้อแก้ปัญหาที่พบจริง เปิดไฟล์ lab7b\_curriculum.py ส่วนที่ 4 ประกอบ

| กติกา                                    | แก้ปัญหาอะไร                                                                                                                   |
|------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------|
| สกัดทุกวิชา ห้ามหยุดกลางทาง              | โมเดลมักหยุดหลังจากได้ 20-30 รายการ แล้วสรุปว่า "และอื่น ๆ"                                                                    |
| อธิบายความหมายของ ปี=0<br>ภาค=0          | ค่า 0 ไม่ได้แปลว่าปี 0 แต่แปลว่า "วิชาเลือกที่ยังไม่กำหนดว่าจะลงเมื่อไร"<br>ถ้าไม่บอก โมเดลจะเดาปีให้เอง ซึ่งเป็นการแต่งข้อมูล |
| prerequisite ถ้าไม่มีให้ใส่คำว่า "ไม่มี" | ไม่ใช่ null และไม่ใช่ list ว่าง เพราะต้องตรงกับรูปแบบของ ground truth                                                          |
| credits ให้คัดลอกตามที่พิมพ์             | ต้องได้ "3(3-0-6)" ไม่ใช่แค่เลข 3<br>บางวิชาเป็น "3(3-0-6) หรือ 3(2-2-5)" ต้องคงไว้ทั้งสอง                                     |
| จำกัดค่าของ category และ type            | ให้เลือกจากชุดที่กำหนดเท่านั้น ไม่ให้โมเดลคิดชื่อหมวดขึ้นมาเอง                                                                 |
| ★<br>แถวช่องวิชาเลือกเป็นข้อมูลจริง      | รหัสอย่าง 06026xxx หรือ xxxxxxxx ไม่ใช่ขยะ แต่หมายถึง<br>"ให้ไปเลือกวิชาจากกลุ่มนี้มาหนึ่งตัว" ต้องสกัดออกมาด้วย               |
| ห้ามสร้างวิชาที่ไม่มีในเอกสาร            | ป้องกัน hallucination ซึ่งเป็นปัญหาที่ตรวจจบบยากที่สุด                                                                         |

## 4. ส่วนที่ 2 — pipeline text: (15 นาที)

```
cd groupB_curriculum

# เลือกเฉพาะหน้าที่เป็นตารางแผนการศึกษา
python3 lab7b_curriculum.py \
  --input data/DSBA_curriculum.pdf \
  --pages "42-58" \
  --gt gt/DSBA_academic_plan_coop.json \
  --pipeline text \
  --out output/
```

### 4.1 สิ่งที่ต้องบันทึก

| หัวข้อ            | สิ่งที่ต้องรายงาน                                                    |
|-------------------|----------------------------------------------------------------------|
| ชนิดของ PDF       | จำนวนตัวอักษรที่ดึงได้ต่อหน้า และข้อสรุปว่าเป็น digital หรือ scanned |
| เวลาที่ใช้        | เทียบกับ pipeline vlm ในส่วนที่ 4 — ต่างกันกี่เท่า                   |
| จำนวนวิชาที่ได้   | เทียบกับจำนวนใน ground truth                                         |
| ผลของ layout=True | ลองแก้โค้ดเป็น extract_text() ธรรมดา แล้วดูว่าปี/ภาคเพี้ยนไปอย่างไร  |

## 5. ส่วนที่ 3 — รัน baseline ด้วย Tesseract ส่วนนี้ถ้าแลกก่อนหน้าทำเสร็จแล้ว ข้ามไปทำข้อ 6 (ไม่มีการส่งย้อนหลัง)

เราต้องมี "เส้นฐาน" ไว้เปรียบเทียบ ไม่เช่นนั้นจะบอกไม่ได้ว่า LLM ดีขึ้นจริงหรือแค่ดูดี

```
python3 lab7b_curriculum.py \
  --input data/DSBA_curriculum.pdf --pages "42-58" \
  --gt gt/DSBA_academic_plan_coop.json \
  --pipeline baseline --out output/
```

### 5.1 อ่านโค้ดในฟังก์ชัน \_rule\_based\_parse

```
# รหัส 8 หลัก + ข้อความ + หน่วยกิตรูปแบบ N(N-N-N)
row_re = re.compile(r"(\d{8})\s+(\.{3,90}?)\s+(\d{1}[\d\s\-\+])")
```

## 6. ส่วนที่ 4 — รัน pipeline VLM (25 นาที)

### 6.1 รอบที่ 1 — benchmark บนหน้าย่อย

เลือกมา 3 หน้าที่เป็นตารางแผนการศึกษาเต็มหน้า แล้วรันทั้งสาม pipeline บนหน้าชุดเดียวกัน

```
python3 lab7b_curriculum.py \
  --input data/DSBA_curriculum.pdf \
  --pages "42-44" \
  --gt gt/DSBA_academic_plan_coop.json \
  --pipeline all \
  --out out_compare/
```

คำสั่งเดียวนี้จะรันครบทั้ง baseline, text และ vlm ตามลำดับ ใช้เวลารวมประมาณ 10 นาที และสร้างไฟล์

out\_compare/comparison.csv ที่มีผลของทั้งสามทำอยู่ในตารางเดียวกัน

### กฎเหล็กของการเปรียบเทียบ

ทุก pipeline ต้องรันบน "หน้าชุดเดียวกัน" เท่านั้น

### 6.2 รอบที่ 2 — รัน pipeline ที่ชนะ บนหน้าทั้งหมด

เมื่อรู้แล้วว่าทำไหนดีที่สุดสำหรับเอกสารของคุณ จึงรันทำนั้นบนหน้าครบเพื่อเอาข้อมูลไปใช้จริง

```
# สมมติผลรอบที่ 1 พบว่า text ดีที่สุด (ซึ่งมักเป็นเช่นนั้นถ้าเป็น digital PDF)
python3 lab7b_curriculum.py \
  --input data/DSBA_curriculum.pdf \
  --pages "42-58" \
  --gt gt/DSBA_academic_plan_coop.json \
  --pipeline text \
  --out out_full/
```



ไฟล์จากรอบที่ 2 คือสิ่งที่จะใช้ต่อใน Lab ต่อไป



out\_full/pred\_text.json จะกลายเป็นฐานข้อมูลตั้งต้นของระบบถาม-ตอบที่จะสร้างในแล็บถัดไป  
 ให้เก็บไฟล์นี้ไว้ พร้อมบันทึกว่าใช้ pipeline อะไร หน้าไหน และค่าตั้งอะไร  
 คุณภาพของไฟล์นี้เป็นพยานคุณภาพของระบบทั้งหมด ไม่มีเทคนิค RAG ไດแก้ข้อมูลผิดได้

### 6.3 ตรวจสอบผลลัพธ์กลางทาง — ขั้นตอนที่ห้ามข้าม

เมื่อรัน pipeline vlm สคริปต์จะบันทึกไฟล์ `intermediate_vlm.md` ซึ่งคือผลจาก Typhoon-OCR  
 ก่อนถูกส่งเข้าโมเดลข้อความ ให้เปิดดูทุกครั้ง



#### ทำไมต้องดูไฟล์กลางทาง

pipeline vlm มีสองชั้น เมื่อ JSON ออกมาผิด เราต้องรู้ให้ได้ว่าผิดตั้งแต่ชั้นไหน

ถ้าใน Markdown ก็ผิดอยู่แล้ว → ปัญหาอยู่ที่การอ่านภาพ ให้แก้ที่ DPI หรือคุณภาพภาพต้นฉบับ

ถ้าใน Markdown ถูก แต่ JSON ผิด → ปัญหาอยู่ที่การจัดรูป ให้แก้ที่พารามิเตอร์หรือ schema

ให้ดูเป็นพิเศษว่าตารางถูกแปลงเป็น markdown table ครบทุกคอลัมน์หรือไม่

การไล่แก้พารามิเตอร์โดยไม่ดูไฟล์กลางทางก่อน คือการเสียเวลาที่พบบ่อยที่สุด

### 6.4 อ่านสัญญาณเตือนจาก log

สคริปต์พิมพ์ค่าเตือนสองแบบที่ห้ามเพิกเฉย

#### แบบที่ 1 — ผลลัพธ์ถูกตัดกลางคัน

[ชั้น 2/2] จัด JSON ก่อนที่ 2/6 (3 หน้า)

(qwen3:4b: 184.2 วิ, 24,891 ตัวอักษร, 16,384 tokens)

⚠ ผลลัพธ์อาจถูกตัดเพราะชน num\_predict

--> ลดจำนวนหน้าต่อ chunk หรือเพิ่ม num\_predict

แปลว่า JSON ที่ได้อาจไม่ครบ ให้ลด LAB7\_CHUNK เป็น 2 หรือ 1 แล้วรันใหม่ อย่ารายงานตัวเลขจากการรันที่มีค่าเตือนนี้  
 เพราะ Recall จะต่ำโดยไม่ใช้ความผิดของโมเดล

#### แบบที่ 2 — มีหน้าที่ดึงข้อความไม่ได้

ดึงข้อความจาก PDF โดยตรง (ไม่ผ่าน OCR)...

ได้ข้อความ 17,329 ตัวอักษร จาก 3/4 หน้า

⚠ มี 1 หน้าที่ดึงข้อความไม่ได้ (น่าจะเป็นหน้าสแกน)

วิชาในหน้าเหล่านั้นจะหายไป

ถ้าเล่มมีหน้าสแกนปน ให้ใช้ --pipeline vlm แทน

สังเกตตัวเลข "3/4 หน้า" ในบรรทัดแรก ถ้าสองตัวนี้ไม่เท่ากันแปลว่ามีหน้าหายไป



#### ทำไมเล่มหลักสูตรถึงมีหน้าสแกนปน

เนื้อหาหลักมักพิมพ์จาก Word แล้วส่งออกเป็น PDF จึงมีข้อความฝังอยู่

แต่ภาคผนวก เช่น ข้อบังคับหรือประกาศของสถาบัน มักเป็นเอกสารเก่าที่ถูกสแกนแล้วแนบมา

ทำให้ไฟล์เดียวมีทั้งสองแบบปนกัน เรียกว่า "ไฟล์ผสม"

ถ้าหน้าที่หายเป็นภาคผนวกที่ไม่มีรายวิชา ก็ไม่กระทบ

แต่ถ้าเป็นหน้าตารางแผนการศึกษา ต้องเปลี่ยนไปใช้ vlm แล้วรายงานเหตุผลในรายงาน

### 6.5 การทดลองปรับค่า (บันทึกผลลงตาราง)

ทดลองอย่างน้อย 2 ใน 4 หัวข้อ บนหน้าชุดเดียวกันทุกครั้ง แล้วบันทึกว่าค่า CER และ Recall เปลี่ยนไปอย่างไร

| ตัวแปร           | ค่าที่ให้ทดลอง                                | สมมติฐานที่ต้องตรวจสอบ                                                               |
|------------------|-----------------------------------------------|--------------------------------------------------------------------------------------|
| ขนาดก้อน         | LAB7_CHUNK=1, 3, 6                            | ก้อนเล็กลงช่วย Recall ไหม แล้วเสียอะไรไป<br>(ดู CER ของ attribute "หมวดวิชา" ประกอบ) |
| ความละเอียดภาพ   | LAB7_DPI=100, 150, 300                        | ยิ่งละเอียดยิ่งแม่นยำเสมอหรือไม่<br>เวลาที่ใช้เพิ่มขึ้นกี่เท่า                       |
| ขนาดโมเดลข้อความ | qwen3:1.7b เทียบ<br>qwen3:4b                  | โมเดลใหญ่กว่าช่วยที่การอ่านตัวอักษร<br>หรือช่วยที่การจัดโครงสร้าง                    |
| วิธีดึงข้อความ   | layout=True เทียบกับ<br>extract_text() ธรรมดา | ดู CER ของ attribute "ปี/ภาค" เป็นหลัก<br>เพราะเป็นตัวที่พังเมื่อคอลัมน์ปนกัน        |

# เปลี่ยนค่าโดยไม่ต้องแก้โค้ด — สังเกตว่า --pages เหมือนกันทุกรอบ

```
LAB7_CHUNK=1 python3 lab7b_curriculum.py -i data/DSBA_curriculum.pdf \
  --pages "42-44" -g gt/DSBA_academic_plan_coop.json -p vlm -o out_chunk1/
```

```
LAB7_CHUNK=6 python3 lab7b_curriculum.py -i data/DSBA_curriculum.pdf \
  --pages "42-44" -g gt/DSBA_academic_plan_coop.json -p vlm -o out_chunk6/
```



#### เคล็ดลับการตั้งชื่อโฟลเดอร์

ชื่อไฟล์ผลลัพธ์ตายตัว (pred\_vlm.json เสมอ) ถ้ารันลงโฟลเดอร์เดียวกันซ้ำ รอบใหม่จะทับรอบเก่าทันที

จึงควรตั้งชื่อโฟลเดอร์ให้บอกได้ว่ารอบนั้นทดลองอะไร เช่น out\_chunk1/ out\_dpi300/ out\_qwen17b/

ตอนเขียนรายงาน เปิด comparison.csv ของแต่ละโฟลเดอร์มาต่อกันใน Excel แล้วเติมคอลัมน์ระบุค่าที่ทดลอง

### 6.6 สิ่งที่ต้องบันทึกจากส่วนนี้

| หัวข้อ           | สิ่งที่ต้องรายงาน                                                                     |
|------------------|---------------------------------------------------------------------------------------|
| ตารางเปรียบเทียบ | CER, WER, Recall, F1 ของทั้งสาม pipeline บนหน้าชุดเดียวกัน<br>ระบุให้ชัดว่ารันหน้าไหน |
| เวลาที่ใช้       | เวลาต่อหน้าของแต่ละ pipeline — ตัวเลขนี้อยู่ในบรรทัด "✓ บันทึก ..." ของ log           |
| ไฟล์กลางทาง      | ยกตัวอย่างจาก intermediate_vlm.md ที่แสดงว่า OCR อ่านตารางได้ดีหรือไม่                |
| ข้อสรุป          | pipeline ไหนเหมาะกับเอกสารของคุณ และเพราะอะไร<br>ต้องอ้างตัวเลข ไม่ใช่ความรู้สึก      |

### ✓ ผลที่คาดว่าจะได้ และบทเรียนที่ซ่อนอยู่

ถ้าเล่นหลักสูตรของคุณเป็น digital PDF pipeline text จะชนะแบบขาดลอย

เร็วกว่า vlm ประมาณ 50 เท่า และ CER ต่ำกว่ามาก เพราะไม่มีโอกาสอ่านตัวอักษรผิดเลย

ต้องเข้าใจลักษณะของปัญหาก่อน แล้วจึงเลือกเครื่องมือ ไม่ใช่เลือกเครื่องมือก่อนแล้วหาปัญหาให้มัน

## 7. ส่วนที่ 5 — ตรวจสอบความสอดคล้องภายใน (15 นาที)

### ✓ แนวคิดหลักของส่วนนี้

ใช้ "กฎเชิงโครงสร้าง" แทน

ทุกกฎต่อไปนี้ตรวจได้โดยไม่ต้องมี ground truth ซึ่งเป็นสถานการณ์จริงเมื่อนำระบบไปใช้งาน

### 7.1 กฎ 5 ข้อที่สคริปต์ตรวจให้

| กฎ | เนื้อหา                                              | จับความผิดพลาดแบบใด                             |
|----|------------------------------------------------------|-------------------------------------------------|
| 1  | รหัสสีขาว 8 ตัว เป็นตัวเลขหรือ x                     | แถวขยะที่ปนมาในตาราง เช่น หมายเลขหรือหัวข้อ     |
| 2  | หน่วยกิตต้องอยู่ในรูป N(N-N-N)                       | อ่านวงเล็บหรือขีดผิด เช่น "3(3~0~6)"            |
| 3  | category และ type ต้องอยู่ในชุดที่กำหนด              | โมเดลคิดชื่อหมวดขึ้นมาจาก                       |
| 4  | ถ้า ปี=0 ภาค=0 ต้องมี<br>flexible_year_semester      | โมเดลเดาปีให้วิชาเลือก หรือลืมหักช่องที่ควรกรอก |
| 5  | prerequisite<br>ต้องอ้างอิงวิชาที่มีอยู่จริงในรายการ | อ่านรหัสวิชาผิด หรืออ่านตักวิชาที่ถูกอ้างอิง    |

### กฎข้อ 5 คือ referential integrity

หลักการเดียวกับ foreign key ในฐานข้อมูล ถ้าวิชา ก. ระบุว่าต้องเรียนวิชา ข. มาก่อน แต่วิชา ข. ไม่มีอยู่ในรายการที่สังกัดได้ แปลว่าเกิดอย่างใดอย่างหนึ่ง

- โมเดลอ่านรหัสของวิชา ข. ผิด ทำให้รหัสไม่ตรงกัน
- โมเดลอ่านตักวิชา ข. ไปทั้งแถว
- โมเดลอ่านรหัส prerequisite ของวิชา ก. ผิด

ทั้งสามกรณีล้วนเป็นข้อผิดพลาดที่เราต้องรู้ และเราจับได้โดยไม่ต้องเปิดเล่มหลักสูตรเทียบเลย

### 7.2 ตรวจสอบผลรวมหน่วยกิตรายภาค

เปรียบเทียบไปกำหนดว่านักศึกษาลงทะเบียนได้ 9–22 หน่วยกิตต่อภาค สคริปต์จึงเตือนเมื่อผลรวมออกนอกช่วงนี้

✗ vlm: 73 วิชา, รวม 118 หน่วยกิต (เฉพาะภาคที่ระบุชุด)

- ภาค 2/2 มีแค่ 6 หน่วยกิต — น่าจะอ่านตักวิชา
- ภาค 3/1 มีถึง 27 หน่วยกิต — น่าจะมีวิชาซ้ำ
- 06026213: prerequisite 06066300 ไม่มีอยู่ในรายการวิชา  
--> อาจอ่านรหัสผิด หรืออ่านตักวิชานั้น

### ✓ รูปแบบที่ควรจดจำและนำไปใช้ในโครงการ

Generate → Verify → Repair → Escalate

สร้างผลลัพธ์ → ตรวจสอบด้วยกฎภายใน → ถ้าไม่ผ่านให้ส่งกลับไปแก้ (พร้อมบอกว่าผิดตรงไหน) →  
 ถ้ายังไม่ผ่านสองรอบให้ส่งคนตรวจ  
 นี่คือรูปแบบที่ใช้จริงในระบบ production ไม่ใช่แค่แบบฝึกหัดในห้องเรียน

### 7.3 ⚠️ ตรวจสอบ ground truth ก่อนใช้

#### ⚠️ อย่าเชื่อว่าไฟล์เฉลยสะอาดเสมอ

ไฟล์ ground truth ของเล่มนี้ถูกส่งออกมาจาก Excel จึงมีสิ่งแปลกปนมา

สิ่งที่ต้องกรองออก (1 แถว):

แถวที่ code เป็นข้อความยาวว่า "หมายเหตุ: คอลัมน์ ..." และ field อื่นเป็น null ทั้งหมด

สิ่งที่ห้ามกรองออกเด็ดขาด (9 แถว):

แถวที่ code เป็น 06026xxx, 9064xxxx, xxxxxxxx

เพราะเป็น "ช่องวิชาเลือก" ซึ่งเป็นข้อมูลจริงในแผนการศึกษา

ถ้าใช้เกณฑ์ "รหัสต้องเป็นตัวเลข 8 หลัก" กรอง จะลบข้อมูลจริงไป 9 แถว

ทำให้ตัวหารเล็กลง แล้วได้ค่า recall สูงเกินจริง

สคริปต์ใช้เกณฑ์เดียวคือ "ต้องมีชื่อวิชาภาษาไทย" ซึ่งเป็นเกณฑ์ที่แคบที่สุดเท่าที่จะทำได้ ดูฟังก์ชัน `clean_gt` ในส่วนที่ 11 ของสคริปต์

## 8. ส่วนที่ 6 — วัดผลด้วย CER และ WER (18 นาที)

### 8.1 รันครบทุก pipeline

```
python3 lab7b_curriculum.py \  
  --input data/DSBA_curriculum.pdf --pages "42-58" \  
  --gt gt/DSBA_academic_plan_coop.json \  
  --pipeline all --out output/
```

สคริปต์จะสร้างไฟล์ output/comparison.csv ให้นำไปเปิดใน Excel แล้วสร้างกราฟแท่งเปรียบเทียบ CER แยกตาม attribute และ pipeline